Technical University of Crete
School of Electrical and Computer Engineering
Course: Optimization
Exercise Projected Gradient (Bonus) (50/1000)
Report Delivery Date: 04/12/2023
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

In this exercise, we used the projections onto simple convex sets and we developed projected and accelerated protected gradient algorithms for constrained quadratic problems. We have the function $f : \mathbb{R}^n \to \mathbb{R}$ defined as

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{P}\mathbf{x} + \mathbf{q}^T\mathbf{x}$$

with $\mathbf{P} \in \mathbb{S}^n_{++}$ and $\mathbf{q} \in \mathbb{R}^n$. Let $L = max(eig(\mathbf{P}))$ and $l = min(eig(\mathbf{P}))$ and consider the quadratic problem

$$min_{x \in \mathbb{S}} f(x)$$

We distinct the following four cases for the set $\mathbb{S}$.

a. $\mathbb{S} = \mathbb{R}^n_+$ First, we were asked to solve the problem via *cvx* and compute the optimal x_* and the value $f(x_*) = p_*$. The code is given below:
Code:

```
% 1a
cvx_begin
variable x(n)
minimize (f_x(x,P,q))
subject to
x >= 0;
cvx_end
x_optimal = x;
```

Then, we proceeded to solve the problem using the projected gradient algorithm. We computed the projection for the particular x and using the form and the terminating condition that was indicated we implemented the projected gradient algorithm, For this implementation the results are the following:
Code:

```
function [x,fun_val,f_val,all_x_kb,iter]=projected_gradient(x0,P,q,n)
x=x0;
gradf = grad_f(x0,P,q);
fun_val=f_x(x0,P,q);
iter=0;
all_x_kb = zeros(n,iter);
f_val = zeros(1, iter);
thresh = 0.000001;
step_size = 1;
```

```
L = max(eig(P));
    while(1)
        iter = iter+1;
        xk_plus = projection_a((x-(1/L)*gradf));

        if (norm(xk_plus - x)/(norm(xk_plus)+thresh/100) < thresh)
            fun_val = f_x(x,P,q);
            break;
        else
            x = xk_plus;
            fun_val = f_x(x,P,q);
            gradf = grad_f(x,P,q);
            all_x_kb(:,iter) = x;
            f_val(iter) = f_x(x,P,q);
        end
    end
    fprintf('\nOptimal value f(x) = %f \n', fun_val);
    fprintf('\nIterations needed for projected algorithm = %d \n',iter);
end
```

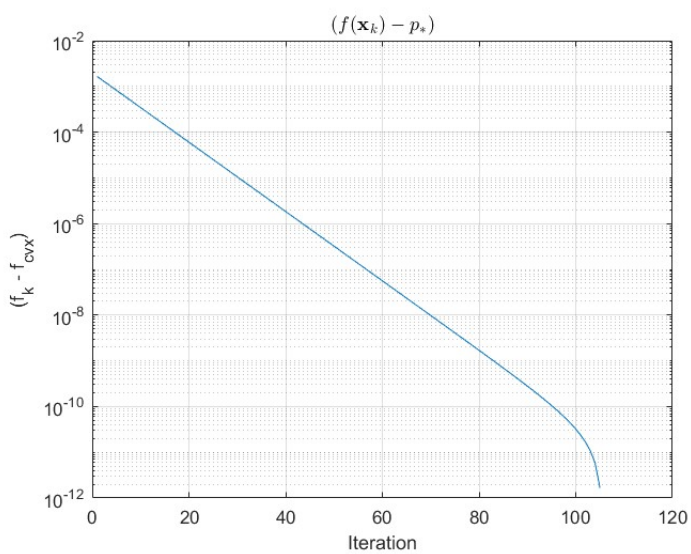


Figure 1: $f(\mathbf{x}_k) - p_*$ for projected gradient algorithm

Then, starting form the same initial point x_0 as before we solved the same problem with the *accelerated* projected algorithm with the indicated forms.

Code:

```
function [x,fun_val_x,f_val,all_x_kb,iter]=
accelerated_projected_gradient(x0,y0,P,q,n)
x=x0;
y = y0;
gradf_x = grad_f(x0,P,q);
fun_val_x=f_x(x0,P,q);
gradf_y = grad_f(y0,P,q);
fun_val_y=f_x(y0,P,q);
iter=0;
all_x_kb = zeros(n,iter);
f_val = zeros(1, iter);
thresh = 0.000001;
L = max(eig(P));
l = min(eig(P));
step_size = 1/L;
b = (sqrt(L)-sqrt(l))/(sqrt(L)+sqrt(l));

while((norm(projection_a(y-step_size*gradf_y)-x))/
(norm(projection_a(y-step_size*gradf_y))) > thresh)
    gradf_x = grad_f(x,P,q);
    fun_val_x = f_x(x,P,q);
    gradf_y = grad_f(y,P,q);
    fun_val_y = f_x(y,P,q);
    x_k = x;
    x = projection_a(y-step_size*gradf_y);
    y = x + b*(x-x_k);
    iter=iter+1;
    all_x_kb(:,iter) = x;
    f_val(iter) = f_x(x,P,q);
end
fprintf('\nOptimal value f(x) = %f \n', fun_val_x);
fprintf('\nIterations needed for accelerated projected algorithm = %d \n',iter);
end
```

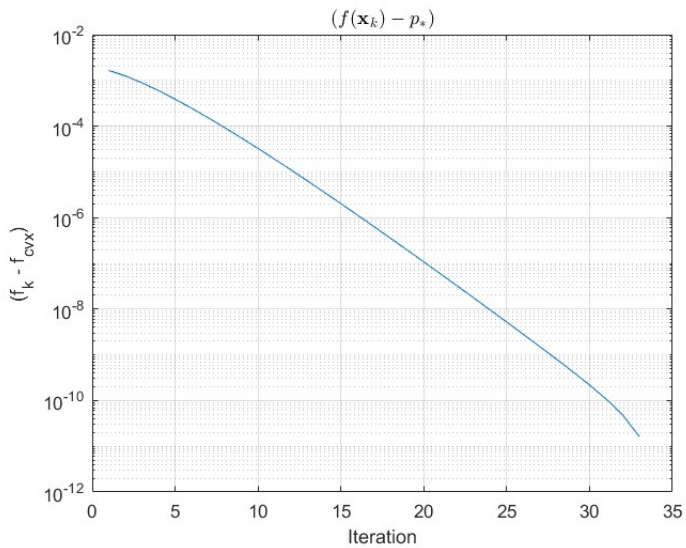


Figure 2: $f(\mathbf{x}_k) - p_*$ for accelerated projected gradient algorithm

Finally, we put the two semilogies plots in the same figure with different colors, so we can compare the efficiency of the two algorithms.

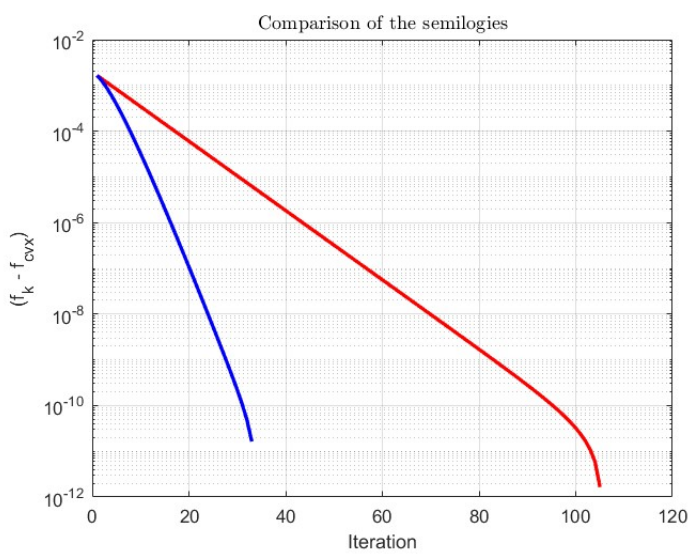


Figure 3: Comparison of the algorithms

We observe that the accelerated gradient algorithm (blue) is much faster and has a steeper fall than the projected. The iterations for the accelerated (for n=2) are a lot less than those of the projected, meaning it reaches the optimal point in less time.

b. Moving on, we are given now the set $\mathbb{S} = \mathbf{x} \in \mathbb{R}^n | \mathbf{a} \leq \mathbf{x} \leq \mathbf{b}$. Following the same procedure as above, we implemented both algorithms and observed the following results. The only thing that changed, when it comes to the algorithms' codes, is that now, and for every set that is going to follow, we have different projection. So, for every case, we have a different function for computing the projection. First, we see the solution that *cvx* gave to the problem:

Code:

```
cvx_begin
variable x(n)
minimize (f_x(x,P,q))
subject to
a <= x <= b;
cvx_end
x_optimal = x;
```

The result was:

```
Status: Solved
Optimal value (cvx_optval): +1.08464


Optimal value f(x) = 1.084639

Iterations needed for projected algorithm = 7

Optimal value f(x) = 1.084639

Iterations needed for accelerated projected algorithm = 5
```

Figure 4: cvx solution

The comparison of the two algorithms is given below:

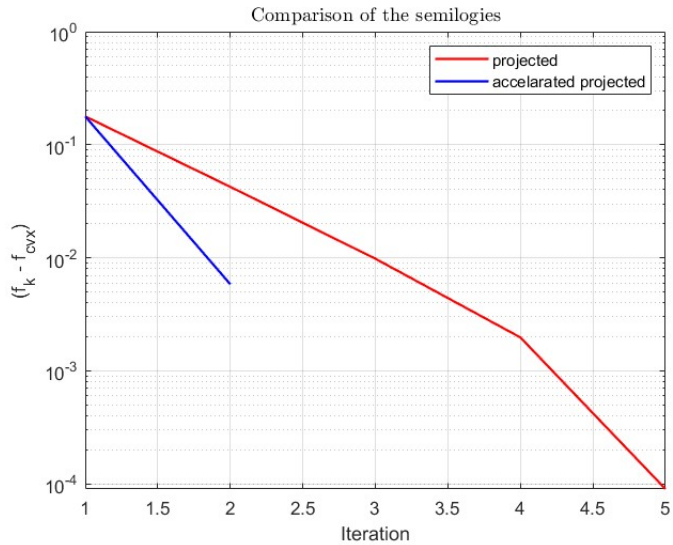
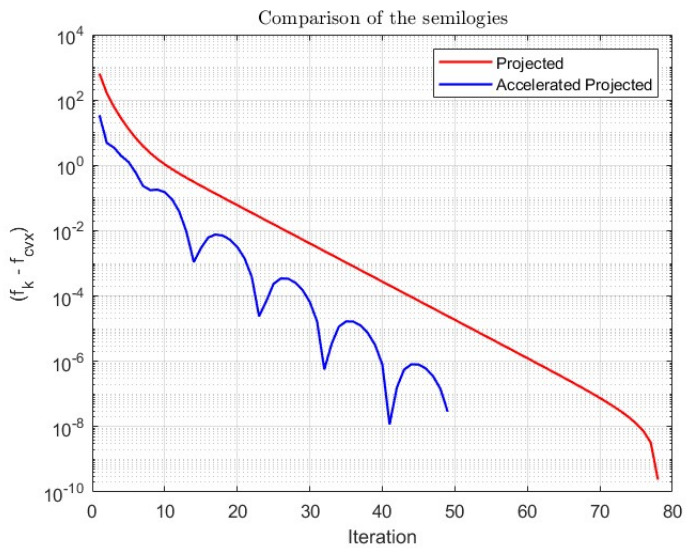


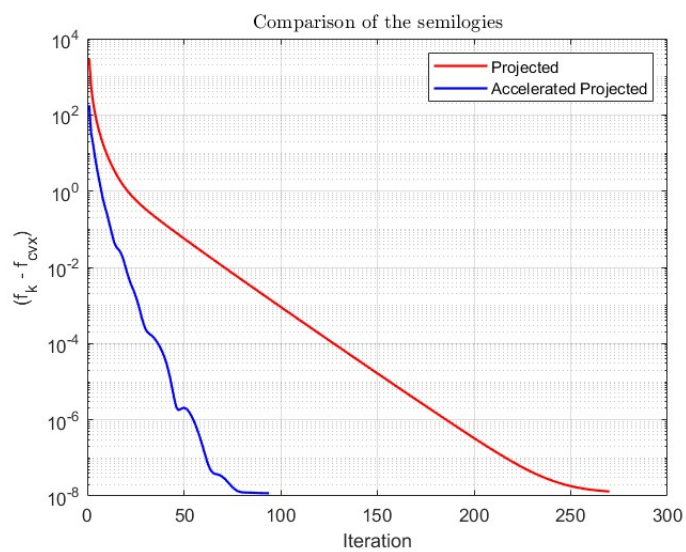
Figure 5: The two gradient algorithms

Again, it is obvious that the accelerated algorithm does much less iterations, in order to reach the optimal result.

c. Next, we have $\mathbb{S} = \mathbf{x} \in \mathbb{R}^n | \mathbf{A}\mathbf{x} = \mathbf{b}$, for $\mathbf{A} \in \mathbb{R}^{p \times n}$, $\mathbf{b} \in \mathbb{R}^p$. For this set, was necessary to define $\mathbf{A}$ and $\mathbf{b}$ and we, also, gathered different results for different values of n and p. Again, we implemented the two algorithms. In that case there was an increase to the number of iterations that each algorithm has to make and that increase gave us the opportunity to observe certain oscillations in the plot of the accelerated algorithm. This happens because the gradient projected algorithm is a descent algorithm, which means that the objective function is monotonically decreasing, while accelerated algorithm is not descent which leads to the oscillation of the objective value. The results were the following (n=20 and p=10), where we can observe what was mentioned:



For n=100 and p=20 we got:



As we see here, the oscillations for the accelerated algorithm are not so many.

d. The final set we examined was $\mathbb{S} = \mathbf{x} \in \mathbb{R}^n | \parallel \mathbf{x} \parallel_2 \leq c$, $\mathbf{x} \in \mathbb{R}++$. Following the same procedure we derived the following results:

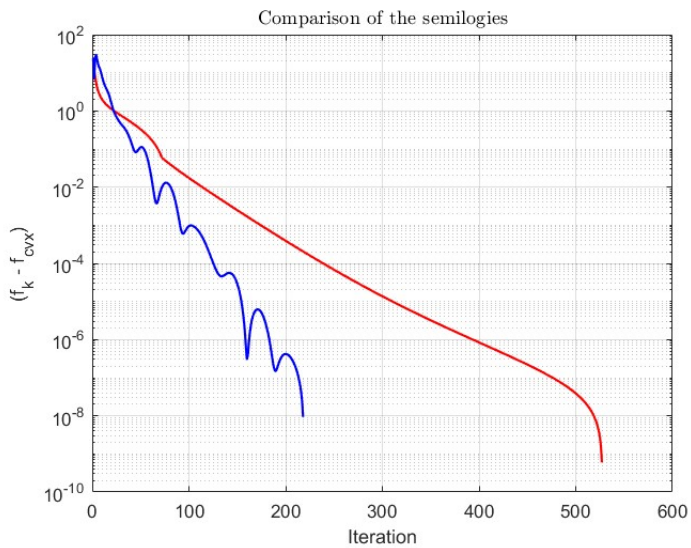


Figure 6: The two algorithms for the fourth set, n=20

The results were expected and once again the accelerated algorithm is faster and more efficient than the projected gradient algorithm.